Classic Friends

Technical handbook · Version 1.0

Installation & Hosting Guide

From a zip file on your laptop to a live app with its own domain and HTTPS — every step, every command, two ways to do



For the owner or whoever sets up the server
Time needed about 1 hour (plus DNS waiting time) · **Cost** from about $6 / month

Contents

1 How hosting works 2

2 Before you start 4

3 Your domain name and DNS 5

4 Path A — Render (no server to manage) 6

5 Path B — Your own Ubuntu server 7

6 Going live: the website and first-day settings 9

7 Keeping it running and secure 10

8 Growing up: payments, sign-in, database, voice at scale, app stores 11

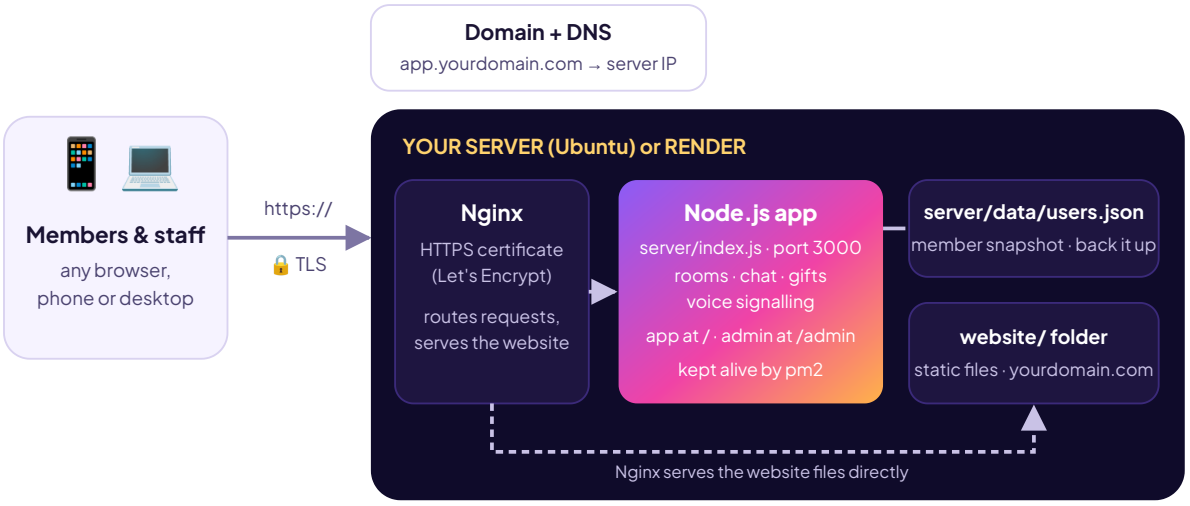
9 Troubleshooting 12

10 Command cheat sheet 13

CHAPTER 1

How hosting works

Classic Friends is one Node.js program. Put it on a computer that is always on, give that computer a name on the internet and a padlock (HTTPS), and the app, the admin console and the website are all live.



1.1 The pieces

Piece	What it is	You get it from
Domain name	The address people type, e.g. `classicfriends.app`. We use `yourdomain.com` for the website and `app.yourdomain.com` for the app and admin console.	A registrar: Namecheap, GoDaddy, Hostinger, Cloudflare (~$10–15 per year)
Server	An always-on computer running the Node.js program.	Path A: Render (managed) · Path B: a VPS from DigitalOcean, Hetzner, AWS Lightsail or Hostinger (~$5–12 per month)
HTTPS certificate	The padlock. Browsers only allow microphones on https:// pages, so this is not optional.	Free, automatic (Let's Encrypt via Certbot, or built into Render)
Nginx	The doorman on your own server: receives https traffic, hands it to the app, serves the website files.	Installed by the setup script (Path B only)
pm2	Keeps the app running, restarts it after crashes and reboots, keeps logs.	Installed by the setup script (Path B only)

1.2 Choose your path

PATH A · EASIEST

Render.com

- No server to manage, HTTPS automatic
- Deploys from a GitHub repository
- Starter plan ~$7 / month + $0.25 for a 1 GB disk
- Free plan for testing (sleeps when idle, forgets data)

PATH B · FULL CONTROL

Your own Ubuntu server

- One script installs everything
- Cheapest at scale (~$6 / month)
- Website and app on one machine
- You do updates and backups

ADD-ON

Website elsewhere

- Host the `website/` folder on Netlify or any shared hosting
- Point its buttons at the app domain
- Chapter 6

Typical monthly cost	Amount
Domain (yearly, averaged)	$1
Path A — Render starter + disk	$7.25
Path B — VPS 2 GB RAM (Hetzner / DigitalOcean / Lightsail)	$5–12
HTTPS certificate	$0
Uptime monitoring (UptimeRobot free plan)	$0
Later: Agora / LiveKit voice at scale	pay as you go (~$1 per 1,000 listener-minutes)

🚂 **Railway, Fly.io, Google Cloud Run, Docker…**
Any host that runs Node or Docker works the same way as Path A. The project root has a `Dockerfile` and `docker-compose.yml`; Railway and Fly.io detect the Dockerfile automatically. Set the same three environment variables (`ADMIN_TOKEN`, `BOTS`, `START_COINS`) and attach a persistent volume at `/app/server/data`.

CHAPTER 2

Before you start

Ten minutes of preparation saves an hour of confusion later.

- [ ] **The project folder** `classic-friends/` (the zip you received), unzipped on your computer.
- [ ] **An email address** for certificates and provider sign-ups.
- [ ] **A payment card** for the domain and the host.
- [ ] **For Path A:** a free GitHub account (github.com). Render deploys from there.
- [ ] **For Path B:** a terminal — on Windows open **PowerShell**; on Mac open **Terminal**. Both already include the `ssh` and `scp` commands we use.
- [ ] **Decide your names.** This guide uses `yourdomain.com` for the website and `app.yourdomain.com` for the app + admin. Replace them with yours everywhere.

> 💡 **Test locally first**
> Run `npm install` and `npm start` on your computer and open http://localhost:3000. If it works there, it will work on the server — the server runs exactly the same program.

2.1 What the project folder must contain on the server

Folder / file	Needed on the server?
`server/` `shared/` `app/` `admin/` `website/` `package.json`	Yes — this is the app
`deploy/`	Yes for Path B — the setup script, Nginx configs, pm2 config, backup and update scripts
`render.yaml` `Dockerfile` `docker-compose.yml`	Yes for Path A and Docker hosts — they must stay at the top level of the folder
`node_modules/`	No — created on the server by `npm install`. Do not upload it (it is large)
`docs/` `dist/`	Optional — documentation and the single-file demo
`.env`	Created on the server with your secrets (Chapter 5). Never put it in a public GitHub repository

CHAPTER 3

Your domain name and DNS

Buy the name, then tell the internet where your server is. DNS changes can take from a few minutes to a few hours to spread.

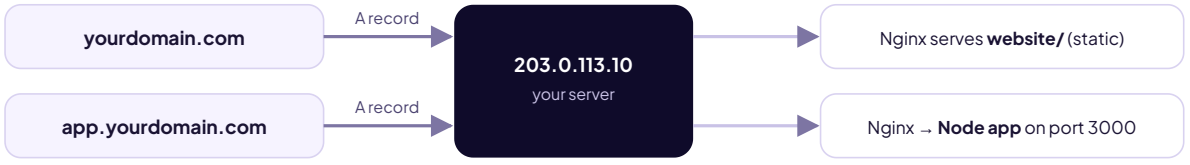
3.1 Buy a domain

1. Go to a registrar (**Namecheap**, **GoDaddy**, **Hostinger**, **Cloudflare**) and search for the name you want; `.com` is the safest choice.
2. Buy one year with auto-renew on. Skip the add-ons (hosting, email, "premium DNS") — you do not need them.
3. In the registrar's dashboard open **DNS settings** (also called **Manage DNS** / **DNS records**) and add the records below.

3.2 DNS records

Leave TTL on "Automatic". "Host" is sometimes called "Name"; `@` means the domain itself. Delete any placeholder `A` or `CNAME` records for `@` and `www` that the registrar created.

Path	Type	Host	Value
B — own server	A	`@`	your server's IP address, e.g. `203.0.113.10`
	A	`app`	the same IP address
	CNAME	`www`	`yourdomain.com`
A — Render	CNAME	`app`	the address Render shows, e.g. `classic-friends.onrender.com`
	A / CNAME	`@` and `www`	wherever the website lives (Render static site, Netlify, or the same service)



3.3 Check that DNS is working

```
# On your computer (Windows PowerShell or Mac Terminal)
nslookup app.yourdomain.com
Name:    app.yourdomain.com
Address: 203.0.113.10          ← must show YOUR server's IP
```

Or check on **dnschecker.org**. Do not run the HTTPS step of the setup until this shows your IP.

> ☁️ **Using Cloudflare?**
> The orange-cloud proxy works, but turn on **WebSockets** (Network settings) and set SSL/TLS to **Full (strict)** once your certificate is installed; until then keep the cloud grey (DNS only).

CHAPTER 4

Path A — Render (no server to manage)

Render runs the Node program for you, gives it HTTPS and restarts it when needed. You push code to GitHub; Render deploys it. About 30 minutes.

4.1 Put the project on GitHub

1. Create a free account at **github.com**.
2. Click **New repository**, name it `classic-friends`, choose **Private**, and create it.
3. Upload the project. Easiest without any tools: on the repository page click **Add file → Upload files**, drag the *contents* of the `classic-friends` folder in (everything except `node_modules`), and click **Commit changes**. With many files this can take a few minutes. Alternatively install **GitHub Desktop**, choose *Add local repository*, and publish.
4. Check the repository shows `package.json`, `render.yaml`, `server/` and `app/` at the top level.

4.2 Create the service on Render

1. Sign up at **render.com** (you can sign in with GitHub) and connect your GitHub account when asked.
2. Click **New → Blueprint**, pick the `classic-friends` repository. Render reads `render.yaml` and shows one web service. (If you prefer the manual way: **New → Web Service**, build command `npm install`, start command `npm start`.)
3. **Plan:** choose **Starter** for a real launch. The blueprint also asks for a 1 GB **disk** mounted at `server/data` so member accounts survive restarts. The **Free** plan is fine for a test, but it sleeps after 15 minutes of inactivity and has no disk.
4. **Environment variables** — the blueprint sets `BOTS=10`, `START_COINS=200` and generates a random `ADMIN_TOKEN`. Open the service → **Environment** to read or change them.
5. Click **Apply / Create**. Watch the **Logs** tab: after a minute you should see *Classic Friends is running* 🎤.
6. Open the address Render gives you (`https://classic-friends-xxxx.onrender.com`). The app loads with HTTPS already working. Sign in, open a room, take a mic — your phone will ask for microphone permission.

4.3 Your own domain on Render

1. In the service open **Settings → Custom Domains → Add** and enter `app.yourdomain.com`.
2. Render shows a CNAME target. Add that CNAME at your registrar (Chapter 3). Render issues the certificate automatically once DNS resolves — usually within minutes.
3. Repeat for the website if you host it on Render too: **New → Static Site**, same repository, **Publish directory** `website`, custom domain `yourdomain.com` (and `www`).
4. Tell the website where the app is: in `website/index.html` uncomment the line `window.CF_APP_URL = "https://app.yourdomain.com"`, commit, and Render redeploys.

4.4 Updating later

Upload or push changed files to GitHub. Render rebuilds and restarts automatically (watch Logs). To roll back, open **Events** and click **Rollback** on a previous deploy.

CHAPTER 5

Path B — Your own Ubuntu server

A small virtual server you fully control. One script installs Node, pm2, Nginx, the firewall and the HTTPS certificate. About 45 minutes, most of it waiting.

5.1 Create the server

1. Sign up with a provider: **DigitalOcean** (Droplet), **Hetzner Cloud**, **AWS Lightsail**, **Vultr** or **Hostinger VPS**. For India and the Gulf pick a region near your members — Bangalore, Mumbai, Bahrain (AWS), or Frankfurt.
2. Create a server with **Ubuntu 22.04 or 24.04**, **2 GB RAM** (1 GB works to start), and either an **SSH key** (recommended, the provider explains how) or a root **password**.
3. Note the server's **public IP address** (e.g. `203.0.113.10`) and add the DNS records from Chapter 3 now, so they propagate while you work.

5.2 Connect to it

```
# Windows PowerShell or Mac Terminal — type yes when asked about the fingerprint
ssh root@203.0.113.10
root@classic-friends:~#          ← you are now typing on the server
```

5.3 Upload the project

Pick whichever you find easiest. The folder must end up at `/opt/classic-friends` on the server.

Option 1 — scp from your computer (one command)

```
# On YOUR computer, from the folder that contains classic-friends/ (not inside it)
# First delete classic-friends/node_modules if it exists — it will be rebuilt on the server
scp -r classic-friends root@203.0.113.10:/opt/classic-friends
```

Option 2 — FileZilla (drag and drop)

Install FileZilla, choose protocol **SFTP**, host = your IP, user `root`, your password or key. Drag the `classic-friends` folder (without `node_modules`) into `/opt/`.

Option 3 — from GitHub

```
# On the server (private repos will ask for a GitHub token as the password)
apt-get update -y && apt-get install -y git
git clone https://github.com/YOUR-USER/classic-friends.git /opt/classic-friends
```

5.4 Run the setup script

```
# On the server. Arguments: app domain · website domain · your email (for the certificate). Takes 5–10 minutes.
bash /opt/classic-friends/deploy/setup-ubuntu.sh app.yourdomain.com yourdomain.com you@email.com
```

The script prints seven numbered steps. This is what each one does:

Step	What happens
1 System packages	Updates Ubuntu and installs curl, git, nginx, unzip and the ufw firewall.
2 Node.js 20 + pm2	Installs the Node runtime and the pm2 process manager.
3 App folder	Runs `npm install` inside `/opt/classic-friends` and creates `server/data` and `backups`.
4 Environment	Creates `/opt/classic-friends/.env` with `PORT=3000`, a **random ADMIN_TOKEN** (printed on screen — save it), `BOTS=10`, `START_COINS=200`.
5 pm2	Starts the app, saves the process list and registers pm2 to start at boot.
6 Nginx	Installs two site configs: `app.yourdomain.com` → the Node app (with WebSocket support) and `yourdomain.com` → the `website/` folder.
7 Firewall + HTTPS	Opens ports 22, 80 and 443 only, then asks Let's Encrypt for certificates and switches both domains to HTTPS with automatic renewal.

```
All done 🎉
  App     → https://app.yourdomain.com
  Admin   → https://app.yourdomain.com/admin/   token: kQ9v2…
  Website → https://yourdomain.com
  Logs    → pm2 logs classic-friends
```

⏳ **If step 7 says the certificate could not be issued**
DNS has not propagated yet. Wait until `nslookup app.yourdomain.com` shows your IP, then run `certbot --nginx -d app.yourdomain.com -d yourdomain.com -d www.yourdomain.com`. Everything else is already in place.

5.5 Verify

1. Open `https://app.yourdomain.com/healthz` — you should see `{"ok":true,…}`.
2. Open `https://app.yourdomain.com` on your phone, sign in, join a room and take a mic. Allow the microphone. Open the same room on a second phone or laptop and talk — you should hear each other.
3. Open `https://app.yourdomain.com/admin/` and sign in with the token from `.env`.
4. Open `https://yourdomain.com` — the website — and press **Open the app**.
5. Reboot test: run `reboot`, wait a minute, open the app again. pm2 brings it back by itself.

CHAPTER 6

Going live: the website and first-day settings

The app is up — now the website, then fifteen minutes of settings before you tell anyone.

6.1 The website on your main domain

The site is plain files in `website/`: host it on the same server (Path B did this) or on any static host.

Where	How
Same server (Path B)	Done by the setup script: Nginx serves `/opt/classic-friends/website` at `yourdomain.com`. To update, upload the new `website/` folder — no restart needed.
Render static site	New → Static Site → same repository → publish directory `website` → custom domain `yourdomain.com`.
Netlify / Vercel	Drag the `website` folder onto app.netlify.com/drop — live in seconds with HTTPS. Add your domain in Site settings.
Shared hosting (Hostinger, cPanel)	Upload the *contents* of `website/` into `public_html` with the file manager.

🔗 **Point the buttons at the app**
When the website and app are on different domains, uncomment the `window.CF_APP_URL` line near the top of `website/index.html` and set it to `https://app.yourdomain.com`.

6.2 First-day settings

1. **Save the admin token** in a password manager. Server: `/opt/classic-friends/.env` · Render: the Environment tab.
2. **Bots.** `BOTS=10` keeps a few rooms alive while the community is small; set `BOTS=0` once real hosts are around every evening. Edit `.env`, then `pm2 restart classic-friends --update-env` (Render: edit the variable).
3. **Welcome coins.** `START_COINS=200` is a sensible gift to new members. Raise it for a launch campaign.
4. **Create a staff account** in the app named "Classic Friends Team" and use it when you join rooms from the admin console.
5. **Banners.** In the admin console set your first three banners (launch offer, weekend event, safety reminder).
6. **Legal pages.** Replace the placeholders in `website/privacy.html` and `website/terms.html` (company name, address, date).
7. **Support email.** Change `supportEmail` and `website` in `shared/catalog.js → BRAND` to your real addresses, then restart.
8. **Monitoring.** Create a free monitor at **uptimerobot.com** for `https://app.yourdomain.com/healthz` — you get an email if the app goes down.

CHAPTER 7

Keeping it running and secure

Path B housekeeping — Render does most of this for you. The hardening list at the end takes ten minutes.

7.1 pm2 — the process manager

```
pm2 status                          # is it running? memory? restarts?
pm2 logs classic-friends            # live log (Ctrl+C to leave)
pm2 restart classic-friends --update-env   # after changing .env
pm2 stop classic-friends            # stop (members see "connection lost")
pm2 start /opt/classic-friends/deploy/ecosystem.config.js && pm2 save
```

7.2 Updating to a new version

1. Back up first (next section). Then upload the new files over the old ones (scp, FileZilla or `git pull`). Do not delete `server/data` or `.env`.
2. Run `bash /opt/classic-friends/deploy/update.sh` — it installs any new dependencies, takes a backup and restarts the app. Members reconnect automatically within a few seconds.

7.3 Backups

Member accounts are in `server/data/users.json`. The script `deploy/backup.sh` copies it with a timestamp into `backups/` and keeps the last 30 copies. Run it daily:

```
crontab -e
# add this line, save, exit
0 3 * * * /opt/classic-friends/deploy/backup.sh
```

Once a week copy the newest backup to your computer (`scp root@203.0.113.10:/opt/classic-friends/backups/users-*.json .`) or turn on your provider's snapshots.

7.4 Server updates, certificates, disk

```
apt-get update && apt-get upgrade -y     # monthly; reboot if it says so
certbot renew --dry-run                  # certificates renew themselves; this checks the
timer
df -h /                                  # disk space (rotate logs: pm2 install pm2-
logrotate)
```

7.5 Security hardening

- [ ] **SSH keys instead of passwords.** Run `ssh-keygen` on your computer, add the public key to the server's `~/.ssh/authorized_keys`, then set `PasswordAuthentication no` in `/etc/ssh/sshd_config` and `systemctl restart ssh`.
- [ ] **fail2ban** blocks repeated login attempts: `apt-get install -y fail2ban` — the defaults protect SSH.
- [ ] **Automatic security updates:** `apt-get install -y unattended-upgrades`.
- [ ] **Rotate the admin token** when someone leaves (edit `.env`, restart) and never use the development default `classicfriends-admin` on the internet.
- [ ] **Backups off the server** (section 7.3) — a hacked or deleted server should never mean lost members.

CHAPTER 8

Growing up: payments, sign-in, database, voice at scale, app stores

None of these are needed on day one. Each one is a contained change; `docs/ARCHITECTURE.md` shows exactly where it plugs in.

When	Add	How
First real money	**Payments** (Razorpay for India, Stripe / Tap / Telr for the Gulf)	Create a checkout for each coin pack. The gateway's webhook calls your server, which calls `engine.recharge(userId, { packId })`. Remove `recharge` from the client `ACTIONS` list in `server/index.js` so coins can only come from verified payments.
Before 1,000 members	**Real sign-in** (phone OTP via MSG91 / Twilio Verify, or Google via Firebase Auth)	After the provider confirms the user, the server calls `engine.login({ id: providerUserId, name, … })`. Never accept an `id` from the client.
Before 1,000 members	**Database** (PostgreSQL on Supabase / Neon / RDS, or MongoDB Atlas)	Persist users, rooms, DMs, moments, families and reports instead of the JSON snapshot. The engine's maps are the seams. Keep daily automated backups on the database provider.
Rooms with 8+ speakers or 30+ listeners	**Agora or LiveKit** for voice	Replace `app/js/voice/webrtc.js` with the SDK: join channel = room id, publish the mic when seated. Also add a **TURN** server (coturn or Twilio) so strict mobile networks connect.
Traffic grows	**Several servers** behind a load balancer	Add the Socket.IO Redis adapter so room events reach every instance; put the database behind them.
Launch marketing	**App stores**	The app is already installable (PWA). For a store listing wrap it with **Capacitor** (iOS + Android) or a Trusted Web Activity (Android). Push notifications via Firebase Cloud Messaging.
Any time	**Analytics**	Add Plausible or Google Analytics to `app/index.html` and `website/index.html`; the admin dashboard already shows the product numbers.

🧩 **Working with a developer**

Hand them `docs/ARCHITECTURE.md` and this chapter. Each item is a few days of work for a Node.js developer; none of them require rebuilding the app.

CHAPTER 9

Troubleshooting

Nearly every hosting problem is one of these.

Symptom	Cause → fix
Browser shows "Welcome to nginx!"	The site config is not enabled. `ls /etc/nginx/sites-enabled` should list `classic-friends-app`; run the setup script again or `ln -s` the config, then `nginx -t && systemctl reload nginx`.
502 Bad Gateway	Nginx is fine but the app is not running. `pm2 status` → if stopped, `pm2 logs classic-friends` shows why (usually a missing `npm install` or a bad `.env` line).
App loads but says "Server not reachable" / demo mode notice	WebSocket is blocked. Check the Nginx config has the `Upgrade` headers (it does in `deploy/nginx-app.conf`); on Cloudflare enable WebSockets.
Microphone does not work on phones	The page is not https://, or the certificate failed. Fix DNS, then run certbot (section 5.4). Also check the phone did not block the mic for the site.
Certbot: "DNS problem" / "connection refused"	DNS not propagated, or port 80 closed. Wait for `nslookup` to show your IP; `ufw status` must show Nginx Full allowed.
"EADDRINUSE: port 3000"	Something else is on port 3000 (often a second copy of the app). `pm2 delete all` then start again from the ecosystem file.
Node version error on `npm install`	Ubuntu's own Node is too old. The setup script installs Node 20; run it again or `node -v` must show 18 or higher.
App stops after a reboot	pm2 startup was not registered: `pm2 startup`, run the command it prints, then `pm2 save`.
Members lost coins after a restart	`server/data/users.json` was deleted or not writable; on Render the disk was not attached. Restore from `backups/`.
Out of memory / slow	1 GB RAM with bots on. Set `BOTS=0`, add 1 GB of swap (`fallocate -l 1G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile`), or resize the server.
Render app is slow to open	Free plan sleeping. Upgrade to Starter, or accept the ~30 s wake-up for testing.
Website shows but "Open the app" goes nowhere	Set `window.CF_APP_URL` in `website/index.html` (section 6.1).

CHAPTER 10

Command cheat sheet

Everything from this guide on one page — replace the domain, IP and paths with yours. Companion documents: User Guide · Admin Console Guide · docs/DEPLOYMENT.md.

Install (Path B)

```
scp -r classic-friends root@203.0.113.10:/opt/classic-friends   # from your computer
ssh root@203.0.113.10
cd /opt/classic-friends                                        # on the server
bash deploy/setup-ubuntu.sh app.yourdomain.com yourdomain.com you@email.com
```

Daily operation

```
pm2 status · pm2 logs classic-friends · pm2 restart classic-friends --update-env
nano /opt/classic-friends/.env               # ADMIN_TOKEN · BOTS · START_COINS · PORT
bash /opt/classic-friends/deploy/update.sh   # after uploading a new version
bash /opt/classic-friends/deploy/backup.sh   # manual backup (cron runs it at 03:00)
curl https://app.yourdomain.com/healthz      # {"ok":true,...}
```

Nginx & HTTPS

```
nginx -t && systemctl reload nginx
certbot --nginx -d app.yourdomain.com -d yourdomain.com -d www.yourdomain.com
certbot renew --dry-run
```

Files that matter

File	What it is
`/opt/classic-friends/.env`	Secrets and settings
`/opt/classic-friends/server/data/users.json`	Member accounts — back it up
`/opt/classic-friends/backups/`	Timestamped copies (last 30)
`/etc/nginx/sites-enabled/classic-friends-*`	Web server configs
`/opt/classic-friends/deploy/`	setup-ubuntu.sh · update.sh · backup.sh · ecosystem.config.js · Nginx configs · Caddyfile
`render.yaml` · `Dockerfile` · `docker-compose.yml`	Project root — for Render and Docker hosts

Settings reference

Variable	Default	Meaning
`PORT`	3000	Port the app listens on (Nginx talks to it)
`ADMIN_TOKEN`	random (script) / classicfriends-admin (dev)	Admin console password
`BOTS`	10 (script) / 70 (dev)	Simulated people; 0 for a real community
`START_COINS`	200 (script) / 1000 (dev)	Welcome coins per new account